

Aardvark and Repokid

Aardvark and Repokid can ensure roles retain only the necessary privileges in large dynamic cloud deployments. These are open source cloud security tools from Netflix, to accommodate rapid innovation and distributed, high-velocity development. They help us to achieve the principle of least privilege access without impacting performance.

All public cloud providers have identity and access management (IAM) services that help to create granular policies for users; however, the custom granular policies add complexity, which can make it more difficult for developers to create a product. Lacking the necessary permission makes the application fail, and excess permissions introduce security breaches and compliance issues.

Aardvark uses Phantoms to log into the cloud provider's console and retrieve Access Advisor data for all of the IAM roles in an account. It stores the latest Access Advisor data in a DynamoDB database and exposes a RESTful API. Aardvark supports threading to retrieve data for multiple accounts simultaneously, and refreshes data in less than 20 minutes.

The DynamoDB table has data about policies, the count of permissions (total and unused), whether a role is eligible for repo or if it is filtered, and when it last removed the unused permissions (repo feature) of each role, as explained in Figure 2. Repokid uses the data stored in DynamoDB with the services not used by a role and removes permissions that were not used.

Once a role has been sufficiently profiled, Repokid's repose feature revises the inline policies attached to a role to exclude unused permissions. Repokid also maintains a cache of previous policy versions in case a role needs to be restored to a previous state. The repo feature can be applied to a single role but is more commonly used to target every eligible role in an account.

OpenShift

OpenShift is a service which is similar to Container-as-a-Service for on-

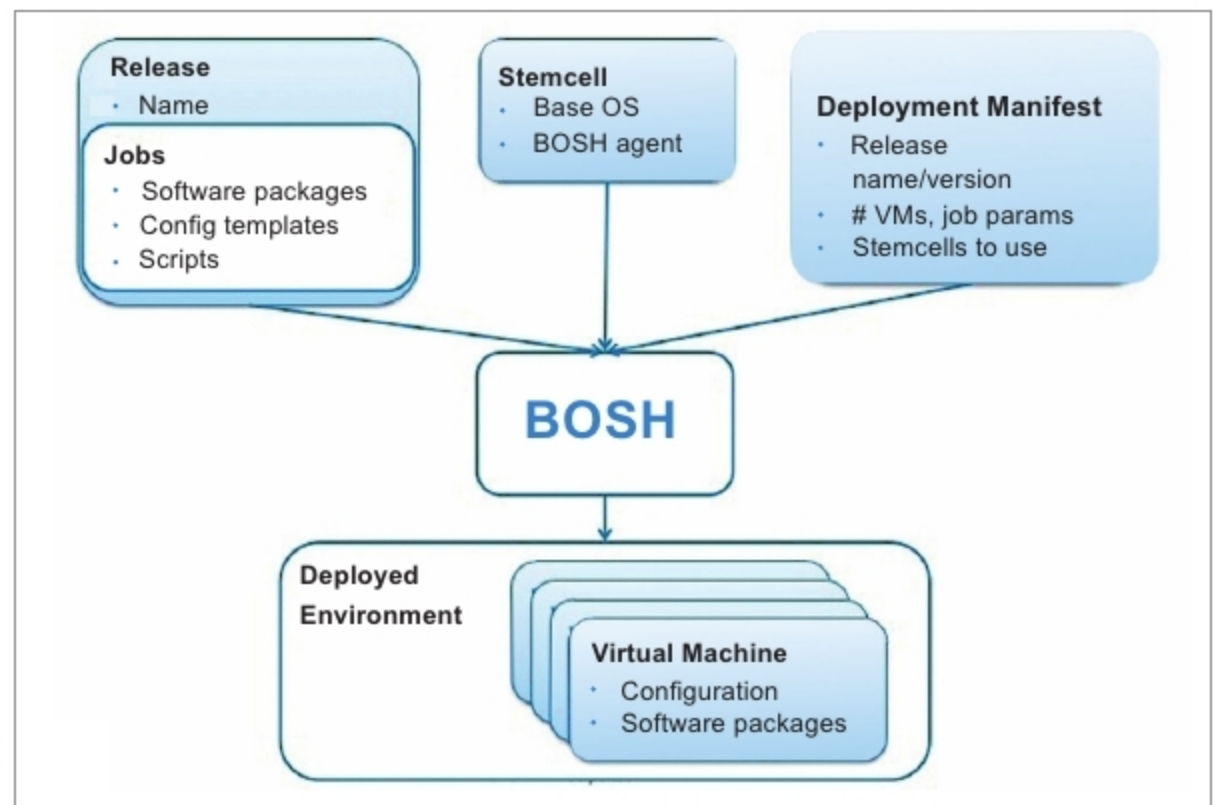


Figure 4: BOSH deployment

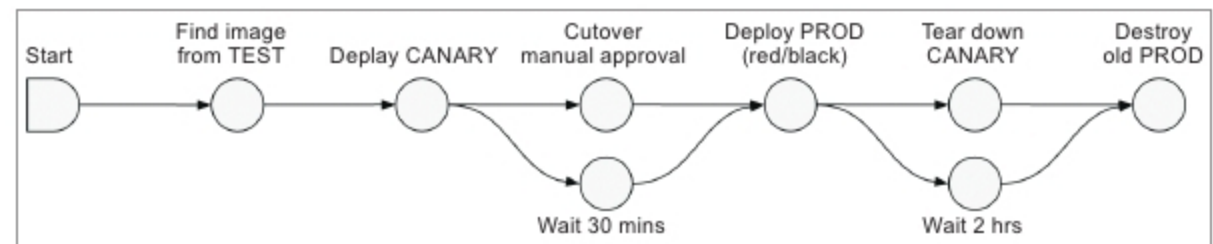


Figure 5: The Spinnaker deployment pipeline

premise and cloud platforms. It is built around Docker containers orchestrated and managed by Kubernetes on a foundation of Red Hat Enterprise Linux. The recent version of V3 comes with many new features. The OpenShift project is backed by Kubernetes, Docker, CoreOS, Framework, Cri-o, Prometheus, etc.

Grouping containers together vastly increases the number of applications that can be brought to OpenShift. This results in the benefit of having a single IP address and sharing a file system. Since OpenShift containers are immutable in nature, the application code, dependent libraries and secrets can be attached to the container at run time. This allows administrators and integrators to separate code and patch from configuration and data. The OpenShift container framework is shown in Figure 3.

OpenShift and Kubernetes are built as sets of microservices working together through common REST APIs to change

the system. These very APIs are available to systems integrators and those same core components can be disabled to allow alternative implementations. OpenShift exposes fine-grained access control on the REST APIs which makes the service integrator's job easy.

In OpenShift v2, an application was a single unit—it consisted of one Web framework and no more than one of any given cartridge type. So an application could have one PHP and one MySQL service, for example, but it could not have one Ruby, one PHP, and two MySQL services. It also could not be a MySQL cartridge by itself. So v2 is helpless when sharing a MySQL instance across two applications.

In OpenShift v3, you can link any two arbitrary components together. If one component can export environment variables and the second component consumes values from those environment variables, you can link together any two components without having to change the images they are based on. So the best